Model Context Protocol (MCP) y JSON-RPC



📋 Agenda de la Presentación

- ¿Qué es JSON-RPC?
- Conceptos Básicos de MCP
- Estructura de Mensajes JSON-RPC
- Entidades en la Arquitectura MCP
- Flujo de Comunicación Completo
- Ejemplo Práctico: Escribir un Archivo
- Ventajas y Beneficios
- Aplicaciones en Ciencia de Datos

🤔 ¿Qué es JSON-RPC?

JSON-RPC = **Remote Procedure Call** usando **JSON** como formato

→

Analogía Simple

→

FUNCIÓN LOCAL:
let resultado = sumar(5, 3)

FUNCIÓN REMOTA:
let resultado = llamarRemoto('sumar', [5, 3])

→

Características Principales

↓

Estándar: RFC 4627 y JSON-RPC 2.0

→

Simple: Basado en JSON (fácil de leer)

→

Interoperable: Funciona con cualquier lenguaje

→

Bidireccional: Cliente ↔ Servidor

🧩 ¿Qué es MCP (Model Context Protocol)?

- **Definición**
- **MCP** es un protocolo que permite a **Claude AI** acceder a herramientas externas de forma segura y estructurada.
- **¿Por qué es importante?**
- Claude puede **leer archivos** de tu computadora
- Claude puede **escribir documentos**
- Claude puede **acceder a APIs**
- Claude puede **procesar datos locales**
- **En Ciencia de Datos**
- Análisis de datasets locales
- Generación automática de reportes
- Procesamiento de archivos CSV/Excel
- Creación de scripts automatizados

📨 Estructura de Mensajes JSON-RPC

- **Mensaje REQUEST (Solicitud)**
- ```
  {
    "jsonrpc": "2.0",
    "method": "write_file",
    "params": {
      "path": "archivo.txt",
      "content": "Hola mundo"
    },
    "id": 1
  }
  ```
- **Mensaje RESPONSE Exitoso**
- ```
  {
    "jsonrpc": "2.0",
    "result": "File written successfully",
    "id": 1
  }
  ```
- **Mensaje RESPONSE Error**
- ```
  {
    "jsonrpc": "2.0",
    "error": {
      "code": -32602,
      "message": "Invalid params"
    },
    "id": 1
  }
  ```
  ```

🏗️ Entidades en la Arquitectura MCP

- **Claude AI** (Anthropic Servers)
- Genera solicitudes de herramientas MCP
- Procesa respuestas
- Mantiene contexto de conversación
- **Claude Desktop** (Tu Máquina)
- **Traductor JSON-RPC**
- Maneja comunicación entre Claude y servidores MCP
- Gestiona configuración
- **MCP Server** (Proceso Node.js)
- Implementa herramientas específicas
- Ejecuta operaciones reales
- Ejemplo: `server-filesystem`
- **Sistema Operativo**
- Acceso real a disco
- Operaciones de archivos
- Recursos del sistema

🔄 Flujo de Comunicación Detallado

- **Paso a Paso**
- `Claude: "Escribe este análisis en un archivo CSV"`
- ```
  {
    "jsonrpc": "2.0",
    "method": "filesystem/write_file",
    "params": {
      "path": "C:/datos/analisis.csv",
      "content": "nombre,edad,ciudad\nJuan,25,Buenos Aires"
    },
    "id": 42
  }
  ```
- **1. Solicitud Claude 2. Traducción Desktop 3. Recepción Servidor**
- Parsea JSON-RPC
- Extrae método: `filesystem/write_file`
- Extrae parámetros: `path`, `content`
- `fs.writeFileSync(path, content)`
- ```
  {
    "jsonrpc": "2.0",
    "result": {
      "success": true,
      "bytes_written": 45
    },
    "id": 42
  }
  ```
- `Claude: "✅ Archivo guardado exitosamente en C:/datos/analisis.csv"`
- **4. Ejecución 5. Respuesta Servidor 6. Confirmación Claude** ---

💡 Ejemplo Práctico: Análisis de Datos

- **Escenario**
- Queremos que Claude analice datos de una encuesta y genere un reporte.
- **Comando de Arranque MCP**
- `npx -y @modelcontextprotocol/server-filesystem`
- **Proceso Completo**
- **1. NPX descarga el paquete 2. Lanza proceso servidor 3. Establece comunicación stdio**
- **Ciclo de Vida**
- Claude Desktop lee configuración
- Ejecuta comando npx
- NPX descarga paquete si es necesario
- Lanza servidor como proceso hijo
- Establece comunicación JSON-RPC
- Servidor queda escuchando requests
- Al cerrar Claude Desktop, mata proceso

🚀 Ventajas de JSON-RPC en MCP

- ✅ **Estándar y Confiable**
- Protocolo bien definido
- Ampliamente adoptado
- Documentación extensa
- ✅ **Simplicidad**
- Basado en JSON (legible)
- Fácil de debuggear
- Estructura clara
- ✅ **Interoperabilidad**
- Cualquier lenguaje puede implementarlo
- Multiplataforma
- Flexible
- ✅ **Bidireccional**
- Cliente → Servidor
- Servidor → Cliente
- Notificaciones en tiempo real



Mecanismos de Transporte

- **STDIO** (Standard Input/Output)
- **Uso**: Procesos locales
- **Ventaja**: Simple y directo
- **Caso**: Servidores MCP locales
- **TCP Sockets**
- **Uso**: Red local/remota
- **Ventaja**: Escalable
- **Caso**: Servidores remotos
- **WebSockets**
- **Uso**: Comunicación web
- **Ventaja**: Tiempo real
- **Caso**: Aplicaciones web
- **Named Pipes**
- **Uso**: Windows/Unix IPC
- **Ventaja**: Eficiente localmente
- **Caso**: Comunicación inter-proceso

⚠️ Códigos de Error JSON-RPC Estándar

- | Código | Nombre | Descripción | |--------|--------|-------------| | -32700 | **Parse Error** | JSON inválido | | -32600 | **Invalid Request** | Estructura incorrecta | | -32601 | **Method Not Found** | Método inexistente | | -32602 | **Invalid Params** | Parámetros incorrectos | | -32603 | **Internal Error** | Error del servidor |
- **Ejemplo de Error**
- ```
  {
    "jsonrpc": "2.0",
    "error": {
      "code": -32602,
      "message": "Invalid params: 'path' is required"
    },
    "id": 1
  }
  ```
  ```

🔍 Debugging JSON-RPC

- Herramientas de Diagnóstico
- Logs de mensajes JSON
- Inspección de requests/responses
- Timeouts y reconexiones
- Validación de esquemas
- Ejemplo de Log
-
```
[DEBUG] Sending request:
{"jsonrpc":"2.0","method":"read_file","params":{"path":"d
atos.csv"},"id":1}

[DEBUG] Received response:
{"jsonrpc":"2.0","result":{"content":"nombre,edad\nJuan,2
5"},"id":1}
```

Resumen Ejecutivo

- **JSON-RPC** es el lenguaje que permite:
- Comunicación estructurada
- Llamadas remotas simples
- Manejo de errores robusto
- **MCP** es el protocolo que permite:
- Acceso seguro a herramientas
- Integración con sistemas locales
- Extensibilidad de capacidades
- **Para Ciencia de Datos** significa:
- Automatización de análisis
- Procesamiento local de datos
- Generación automática de reportes
- Integración con herramientas existentes